

HBFlex: A Flexible Memory System for Bridging Fine-Grained LLM States and Coarse-Grained HBF Parallel Execution

Shuzhang Zhong*
Peking University
Beijing, China

Weikai Xu*
HKUST
Hong Kong, China

Yifan Zhou
Peking University
Beijing, China

Tongbin Zhao
Peking University
Beijing, China

Tenghao Zhao
Peking University
Beijing, China

Yifei Kang
Alibaba Group
Hangzhou, China

Cunyun Chang
Alibaba Group
Hangzhou, China

Shu Li
Alibaba Group
Hangzhou, China

Guangyu Sun
Peking University
Beijing, China

Meng Li†
Peking University
Beijing, China
meng.li@pku.edu.cn

Abstract

Large language models (LLMs) require increasing memory capacity to accommodate growing model weights and KV caches. High-Bandwidth Flash (HBF) offers high memory density and aggregate read bandwidth through massive plane-level parallelism, making it an attractive option for LLM serving. However, serving LLMs entirely from HBF introduces three challenges: fine-grained KV reads create placement and access imbalance, incremental writes interfere with foreground reads, and mixed KV lifetimes amplify garbage collection. Hybrid HBM/HBF designs retain HBM to support dynamic KV management, but this allocation reduces the HBF resources available under a fixed packaging budget, limiting aggregate HBF bandwidth.

We present HBFlex, a full-HBF memory system with coordinated optimizations for KV reads, writes, and reclamation. HBFlex balances KV placement and attention accesses to improve plane utilization. It aggregates incremental updates and schedules writeback within sufficiently long compute windows to reduce write-read interference. It also combines lifetime-guided block packing with deferred reclamation to reduce valid-page migration. We evaluate HBFlex through trace-driven simulation across different configurations. HBFlex achieves average throughput speedups of up to 1.58× over FlashAccel and 3.30× over H3, benefiting from higher HBF bandwidth and more efficient management of dynamic KV-cache reads, writes, and erases.

1 Introduction

Large language models (LLMs) and agentic applications [22, 29, 35] are placing increasing pressure on accelerator memory capacity. Model sizes continue to grow, while long contexts [33, 39], multi-turn interactions [7, 37, 40], and concurrent requests increase the amount of key-value (KV) state

that serving systems must retain. For example, DeepSeek-V4-Pro [6] contains 1.6 trillion parameters and supports a one-million-token context window. Preserving useful KV histories also avoids repeated prefill computation across related requests [25, 41]. However, the limited capacity of high-bandwidth memory (HBM) makes it difficult to accommodate both large models and their growing KV caches on a single accelerator. Distributing this state across more GPUs relieves capacity pressure but adds communication and coordination overhead, increasing the cost of serving [3, 17, 23, 42].

High-Bandwidth Flash (HBF) offers a promising path to expanding accelerator-local memory capacity by combining the high density of NAND flash with vertically stacked dies and a wide accelerator interface [14, 18, 26]. Although an individual NAND access is slower than a DRAM access, thousands of independent planes can operate concurrently, allowing HBF to target read bandwidth comparable to HBM with substantially greater capacity density. Its bandwidth advantage therefore depends on exposing sufficient plane-level parallelism and distributing useful work evenly across many planes.

Recent studies have explored hybrid HBM/HBF organizations that capture HBF’s capacity benefits while retaining HBM for state that benefits from low latency and efficient updates. These proposals span *co-located* designs with separate HBM and HBF stacks beside the GPU [15, 31], *cascaded* designs that connect HBF through the HBM base die [9, 21, 31], and *integrated-stack* designs that combine DRAM and flash dies within one stack [38]. A common approach places read-mostly state, such as weights and precomputed shared KV, in HBF while retaining dynamic request KV in HBM [9].

However, under a fixed package budget, retaining HBM comes at the expense of HBF resources. HBM occupies stack locations or die resources that could otherwise accommodate

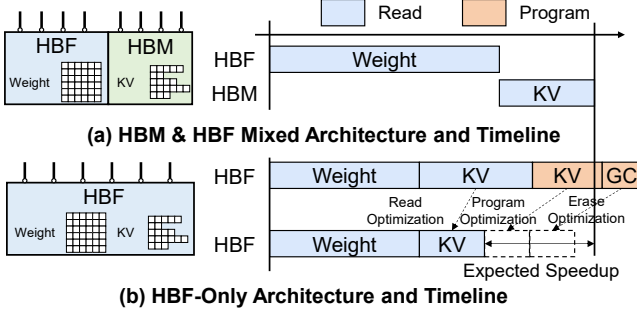


Figure 1. From hybrid HBM/HBF serving to full-HBF serving. (a) A hybrid organization separates weight reads and dynamic KV accesses across HBF and HBM. (b) A full-HBF organization provides more HBF resources, but realizing its performance potential requires coordinated optimization of KV reads, programs, and block erases.

additional HBF planes. Because HBF derives aggregate bandwidth from these planes, allocating fewer resources to HBF limits both its capacity and its available read parallelism. In particular, the bandwidth available for HBF-resident weights remains constrained even when HBM efficiently serves KV accesses, as illustrated in Figure 1(a).

Dedicating the entire memory budget to HBF eliminates this resource trade-off but places dynamic KV reads, writes, and reclamation on the same flash device. Fine-grained accesses can leave much of the nominal bandwidth unused, incremental updates can block subsequent reads, and garbage collection (GC) can introduce additional data movement and consume flash endurance [8, 20]. Consequently, simply moving KV into HBF does not ensure that its additional planes translate into better serving performance [16].

Exploiting a full-HBF organization therefore requires a memory system that reconciles the logical behavior of KV caches with the physical constraints of flash. As Figure 1(b) illustrates, the system must coordinate read, program, and erase operations to preserve the parallelism gained by dedicating more resources to HBF. We identify three challenges that prevent a direct mapping of conventional KV management onto this organization.

Challenge 1: Fine-grained KV cache creates read imbalance. This imbalance arises at two distinct levels. Request completion and reactivation can create **placement imbalance**. They change the active KV working set without necessarily moving its pages, disrupting a previously balanced placement. Balancing new allocations therefore does not ensure balance as request activity changes. Even with balanced placement, fine-grained reads can create **access imbalance**. Each attention invocation reads only a subset of KV pages. PagedAttention [13] and chunked CTA scheduling [1, 2] can select pages concentrated on a few planes.

These collisions require additional read waves while other planes remain idle.

Challenge 2: Incremental KV updates create write-read interference. Incremental KV updates require buffering to form writes that exploit plane parallelism. However, each program occupies its target plane for approximately $75\ \mu\text{s}$ and cannot be preempted. Aggregation alone therefore cannot prevent interference with foreground reads. Interleaved weight and KV reads divide computation into short gaps that may not hide a full program. Deferring writes avoids immediate conflicts, but buffer pressure eventually forces writeback. Efficient writeback must coordinate both write aggregation and program timing.

Challenge 3: Interleaved KV lifetimes create erase amplification. Raw KV writes remain within the estimated endurance budget under our workload assumptions. However, block-level erase requires valid pages to be migrated when KV pages with different lifetimes share a block. Our characterization of DeepSeek-V4-Pro shows an average GC write amplification of approximately 30 $\times$, exceeding the available endurance margins. Isolating requests in dedicated blocks simplifies erase management, but constrains plane-level striping. KV placement must therefore balance read parallelism against lifetime locality.

We present *HBFlex*, a cross-layer memory system that addresses these challenges in a full-HBF organization. HBFlex couples base-die SRAM buffering with co-located prefill and decode execution, allowing prefill computation to provide opportunities for draining decode KV updates. Its system design separates logical reuse granularity from physical striping, coordinates KV placement with attention scheduling, creates and exploits sufficiently long writeback windows, and combines lifetime-aware block packing with deferred reclamation. We evaluate HBFlex using a custom trace-driven simulator that models individual read and write operations, across four LLMs [5, 6, 27, 34], four Alibaba Bailian production traces [30], and SWE-bench workloads [12] with varying concurrency. This paper makes four contributions:

- We characterize the mismatches between dynamic KV management and HBF read, program, and erase operations, distinguishing placement imbalance from access imbalance and identifying the trade-offs among reuse, parallelism, write timing, and reclamation.
- We propose a full-HBF architecture with base-die SRAM and co-located prefill and decode, preserving HBF capacity and plane resources while buffering dynamic KV updates.
- We design three coordinated mechanisms: HBF-aware KV placement and scheduling, window-aware writeback, and lifetime-aware reclamation, enabling efficient KV management while retaining a fine-grained logical interface.

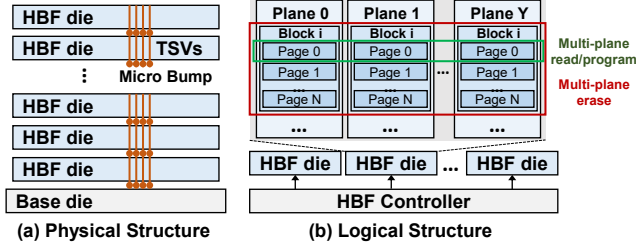


Figure 2. HBF organization: (a) stacked NAND dies and a base die; (b) the die–plane–block–page hierarchy.

- Across the evaluated configurations, HBFlex achieves average throughput speedups of up to 1.58× over FlashAccel and 3.30× over H3, benefiting from higher HBF bandwidth and more efficient KV-cache management. Ablation studies quantify the benefits of read balancing, writeback scheduling, and lifetime-aware reclamation.

2 Background

2.1 High-Bandwidth Flash Basics

High-Bandwidth Flash (HBF) combines NAND flash density and non-volatility with a wide accelerator interface [20]. As illustrated in Figure 2, NAND dies are stacked above a base die that connects to the GPU and manages flash operations. HBF follows a *stack–die–plane–block–page* hierarchy, with each plane providing an independent array datapath and page buffer. Array reads and programs operate on 4-KiB physical pages, while erases operate on blocks. The host interface may return the requested portion of a buffered page.

HBF achieves high aggregate bandwidth through concurrent operations across many planes. Striping pages across planes and host channels exposes this parallelism, but sustaining peak bandwidth requires sufficient outstanding requests and balanced access loads. Accesses to the same plane serialize, so the most heavily loaded plane determines completion time while less-loaded planes become idle.

2.2 LLM Workload

LLM inference consists of *prefill*, which processes prompt tokens in parallel, and *decode*, which generates tokens incrementally while reusing cached KV vectors.

To support attention over this growing token history, Page-dAttention organizes KV state into small *logical KV pages* mapped through a page table to independently allocated physical pages [13, 24], as illustrated in Figure 3. A logically contiguous sequence therefore spans fine-grained pages scattered across memory. At runtime, attention kernels divide the sequence into *work tiles*, or KV chunks, and schedule them across cooperative thread arrays (CTAs). Each CTA follows the page mappings to fetch its KV chunk, computes

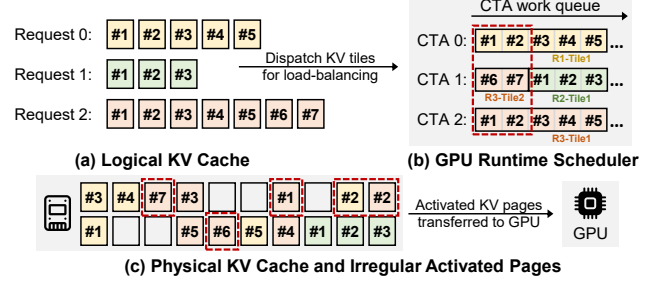


Figure 3. Paged KV-cache storage and chunked attention scheduling across CTAs.

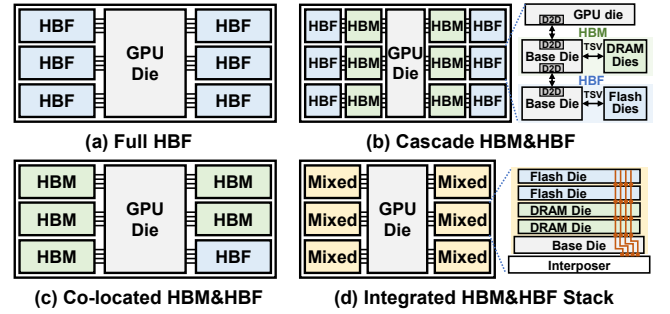


Figure 4. Representative HBF integration architectures.

partial attention results using the query vector, and contributes to a per-request reduction [4, 36]. Consequently, a CTA’s logically ordered work translates into fine-grained reads from scattered, request-dependent physical locations, rather than a single contiguous memory region.

2.3 HBF for LLM Serving

Recent architectures combine HBM and HBF to expand LLM serving capacity, as illustrated in Figure 4. H3 [9] connects HBF through the HBM base die, placing weights and read-only shared KV in HBF while retaining generated KV in HBM. FlashAccel [31] explores both cascaded and co-located organizations, storing weights and KV in HBF while using HBM for buffering and read balancing. Integrated-stack designs [38] instead combine DRAM and flash dies within each stack, expanding capacity while preserving HBM-like interfaces.

These designs retain HBM/DRAM to support dynamic or latency-sensitive accesses. Under a fixed package or stack budget, this allocation limits the resources available for HBF dies and parallel planes, constraining both HBF capacity and aggregate bandwidth. Efficiently managing dynamic KV directly in HBF could reduce this dependence on HBM, but requires reconciling KV access and update patterns with HBF’s page-level operations and block-level reclamation.

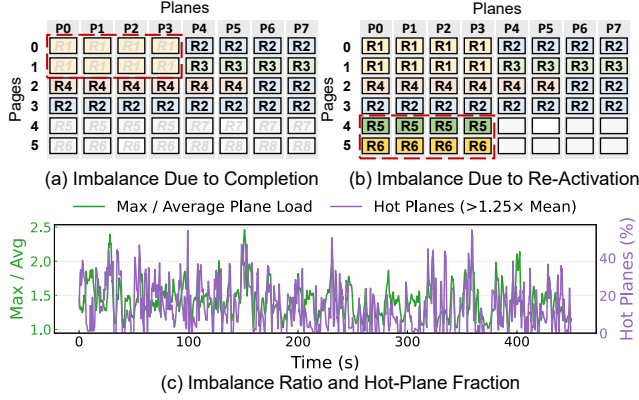


Figure 5. Placement imbalance from (a) request completion and (b) retained-KV reactivation. (c) Imbalance ratio and hot-plane fraction during a 450-s Bailian trace. Hot planes carry more than 1.25 $\times$ the mean load.

3 Motivation

Efficiently accommodating dynamic KV cache in HBF requires aligning KV management with the underlying flash organization. To understand the challenges of such a design, we examine how fine-grained KV accesses and dynamic updates interact with HBF’s coarse-grained, plane-parallel organization.

Our analysis uses an industry-provided HBF configuration with six stacks, 16 dies per stack, and 32 planes per die, totaling 3,072 planes. Each plane has 512 blocks of 512 pages each, with read and program latencies of approximately 4 and 75 μ s [31], respectively. The configured read bandwidth is 488 GB/s per stack, or 2.93 TB/s across six stacks. Realizing this bandwidth requires balanced accesses across planes. Dynamic KV state complicates this requirement through changing active requests and fine-grained reads, while incremental updates and mixed page lifetimes introduce program interference and GC amplification. These behaviors motivate the following three challenges in **KV read, write, and erase behaviors**.

3.1 Challenge 1: Fine-Grained KV Cache Creates Read Imbalance

HBF read latency is determined by the most heavily loaded plane because page reads within a plane serialize. We quantify imbalance as the maximum plane load divided by the average:

$$\rho = \frac{\max_i L_i}{\frac{1}{N} \sum_{i=1}^N L_i}, \quad (1)$$

where N is the number of planes and L_i counts the relevant physical pages on plane i . We measure this load over either the active KV working set or the pages selected by an attention invocation. These two scopes capture placement

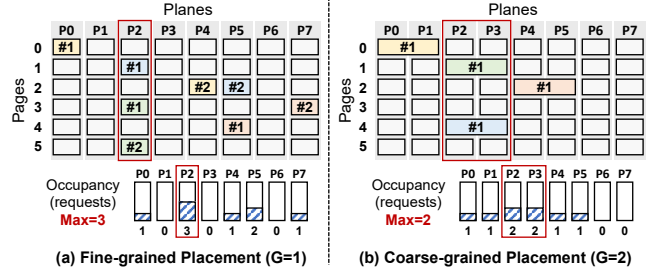


Figure 6. Access imbalance under balanced placement. Maximum plane occupancy decreases from three (a) to two (b) with two-page striping. G denotes physical pages per stripe.

and access imbalance, respectively. A ratio of one indicates perfect balance.

Placement imbalance. Request completion removes KV pages from the active working set without necessarily deallocating them. This can expose imbalance among the remaining requests. Allocating new pages to lightly loaded planes repairs the current imbalance, but retained pages remain at their original locations. Reactivating these pages can overload planes that have since received new allocations, as illustrated in Figure 5(a) and (b).

We use traces from the Alibaba Bailian platform [30] to examine placement imbalance under our HBF configuration. As shown in Figure 5(c), the imbalance ratio and hot-plane fraction exhibit repeated spikes throughout the trace. Balancing the current active set therefore does not ensure balance after subsequent completions and reactivations.

Access imbalance. Even with balanced placement, an attention invocation may access planes unevenly. PagedAttention [13] and chunked CTA scheduling [1, 2] select fine-grained KV subsets whose pages can collide on the same plane. For example, a 12-MiB read contains 3,072 physical pages, enough to access every plane once under ideal placement. Collisions instead force some planes to serve multiple pages sequentially. This requires additional read waves, as illustrated in Figure 6.

Increasing logical-page size can reduce these collisions, but it coarsens reuse. Under the 1-KiB-per-token model, larger logical pages span more physical pages striped across distinct planes. This reduces independent placements and maximum plane occupancy in a 12-MiB read, as shown in Figure 7(a). However, prefix matching reuses only complete logical pages. We define *prefill amplification* as the tokens recomputed with page-granular matching divided by the minimum new tokens under token-granular matching. As shown in Figure 7(b), larger pages increase this amplification by discarding reusable page tails. Tying logical reuse granularity to physical access granularity therefore trades fine-grained reuse for plane-level read parallelism.

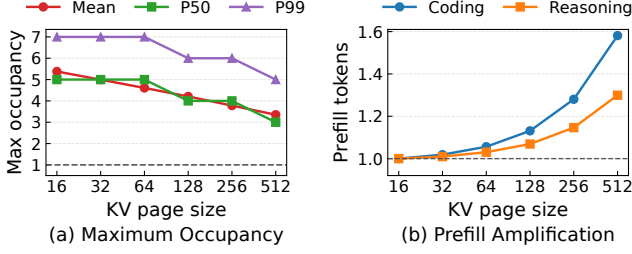


Figure 7. KV-page size trade-off under balanced placement. Larger pages reduce maximum plane occupancy for a 12-MiB read (a), but increase prefill amplification (b).

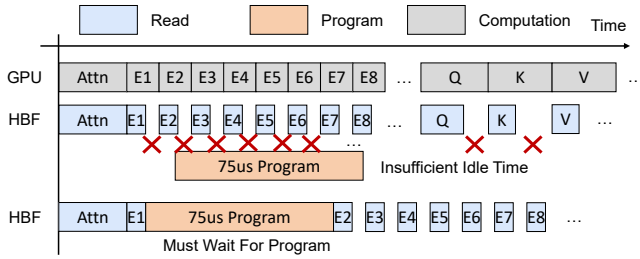


Figure 8. HBF program interference with foreground reads. Short compute gaps cannot hide a full program, causing subsequent reads to the same plane to stall.

3.2 Challenge 2: Incremental KV Updates Create Write-Read Interference

Each generated token adds a small amount of KV state at every layer. Writing these updates immediately underutilizes plane parallelism. Buffering them enables wider writes, but increases buffer occupancy. More importantly, each program occupies its target plane for approximately 75 μ s and cannot be preempted. Even an aggregated write can therefore stall subsequent reads to the same planes.

Hiding this latency requires a sufficiently long interval without reads to the programming planes. However, weight and KV reads are interleaved with computation, dividing potential write windows into short gaps. As illustrated in Figure 8, a program that extends beyond a gap blocks the next conflicting read. Deferring it avoids the immediate stall, but buffer pressure eventually forces writeback.

As shown in Figure 9, prefill computation grows with chunk size, while most individual decode stages remain shorter than one program. This comparison includes computation and KV writes but excludes intervening HBF reads. Smaller writes occupy fewer planes but do not shorten the program latency on each plane. Efficient writeback therefore requires both aggregating updates and scheduling programs within sufficiently long compute windows.

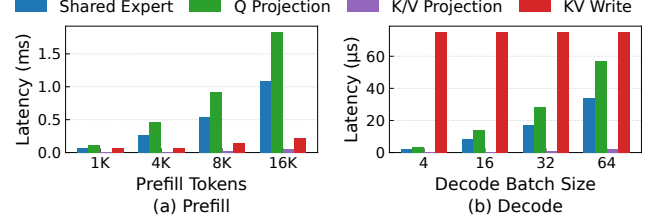


Figure 9. Computation and KV-write latency for DeepSeek-V4-Pro at an ideal 2,000 TOPS and a 75- μ s HBF program latency. (a) Prefill with varying chunk sizes. (b) Decode with varying batch sizes. Intervening HBF reads are excluded.

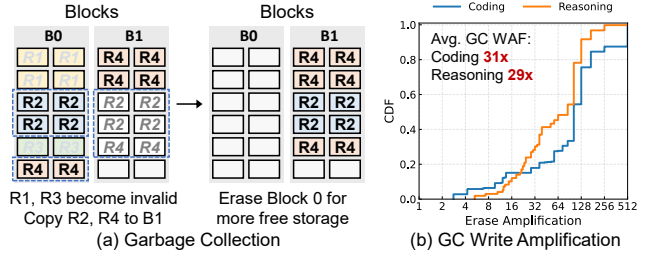


Figure 10. Garbage-collection amplification caused by interleaved KV lifetimes. (a) Valid KV pages must be migrated before block erase. (b) This effect leads to substantial GC amplification, with an average WAF of approximately 30 $\times$ for DeepSeek-V4-Pro.

3.3 Challenge 3: Interleaved KV Lifetimes Create Erase Amplification

Flash endurance is a potential concern when storing dynamic KV cache in HBF [26]. DeepSeek reports approximately 434B new-KV tokens per day. Using 35 KB/token for DeepSeek-V3 [5], this corresponds to 15.19 PB/day system-wide, or 8.37 TB/day per GPU across 1,814 GPUs. At the same token rate and GPU count, Qwen3 [34] would require 22.73 TB/day per GPU with 95 KB/token. A 1-TB HBF with 100K P/E cycles supports approximately 54.8 TB/day over five years. Raw KV writes consume 15.3% of this budget for DeepSeek-V3 and 41.5% for Qwen3 under the same workload assumptions. This leaves room for approximately 6.5 $\times$ and 2.4 $\times$ write amplification, respectively.

The actual problem comes from HBF’s block-granular erase semantics [20]. KV pages from different requests are interleaved in the same blocks but become invalid at different times. As shown in Figure 10(a), when R1 and R3 expire, valid pages belonging to R2 and R4 must first be migrated before the block can be erased, introducing additional writes and erases through garbage collection. As shown in Figure 10(b), the average GC WAF for DeepSeek-V4-Pro reaches approximately 30 $\times$. This exceeds the write-amplification margins estimated above, making lifetime mixing a critical endurance concern.

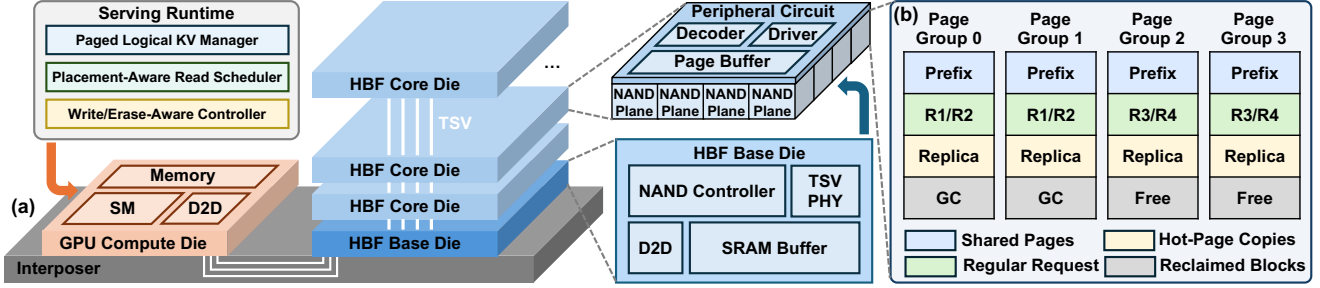


Figure 11. Overview of HBFlex architecture. (a) The GPU compute die connects to the HBF base die through D2D and the base die coordinates TSV-connected NAND core dies. (b) The schematic page-group layout with shared pages (blue), regular requests (green), hot-page copies (yellow), and reclaimed blocks (gray).

A seemingly simple solution is to place each request within dedicated blocks so that its KV cache can be reclaimed together. However, such request-isolated placement severely restricts plane-level striping and, as demonstrated in Challenge 1, creates substantial load imbalance and bandwidth loss. Therefore, KV-cache placement faces a fundamental tension: interleaving requests improves read parallelism, while isolating request lifetimes reduces erase amplification. Efficient HBF-based KV-cache management must jointly address both objectives.

4 HBFlex Architecture

In this work, we propose HBFlex, a cross-layer memory system that accommodates the dynamic KV cache of LLM serving in a full-HBF package while preserving the fine-grained logical interface that serving runtimes expose. HBFlex co-locates prefill and decode on the same GPU-HBF package rather than using prefill-decode disaggregation, allowing prefill computation to provide HBF-free windows for draining decode KV writes. By devoting the package’s entire memory budget to HBF rather than retaining HBM, HBFlex maximizes capacity for the growing KV cache and preserves HBF’s plane-level parallelism, supporting larger batch sizes and higher throughput without the HBF bandwidth loss incurred by a hybrid HBM/HBF package. Section 4.1 develops the rationale for these choices and introduces the base-die SRAM buffer, while Section 4.2 describes the resulting architecture.

4.1 Design Considerations

Full-HBF package. Although flash endurance may appear to limit dynamic KV storage, the workload-level analysis in Section 3.3 shows that raw KV writes remain within the estimated endurance budget. Garbage collection, rather than the raw write volume, is the dominant concern, with measured write amplification of approximately 30× for DeepSeek-V4-Pro. HBFlex therefore dedicates the package to HBF rather than retaining HBM, which maximizes KV-cache capacity and preserves HBF’s plane-level parallelism, supporting larger

batch sizes and higher throughput. The resulting placement, scheduling, write-formation, and reclamation challenges are addressed in Section 5.

Fused prefill and decode. HBF reads are fast, massively parallel, and latency-critical, whereas programs are coarse, non-preemptible, and occupy a plane for about 75 μ s (Section 3.2). Because newly generated KV can be deferred, the scarce resource is a sufficiently long write window rather than aggregate write bandwidth. Prefill-decode disaggregation separates continuous fine-grained decode writes from the compute-heavy prefill intervals that could absorb them. HBFlex therefore co-locates both phases on one GPU-HBF package, buffers decode KV locally, and drains it during prefill when the target planes are off the critical path (Section 5.3).

Base-die SRAM buffer. The base-die SRAM is shared by HBF read prefetching, decode-KV writeback, hot-plane offload, and garbage-collection migration. Its required capacity is determined by the peak simultaneous occupancy:

$$C_{\text{SRAM}}^{\text{req}} = \max_t [C_{\text{WPF}}(t) + C_{\text{KV}}(t) + C_{\text{aux}}(t)], \quad (2)$$

where $C_{\text{WPF}}(t)$ is the live weight-prefetch footprint, $C_{\text{KV}}(t)$ is the buffered decode-KV state, and $C_{\text{aux}}(t)$ accounts for hot-plane offload and garbage-collection migration. For DeepSeek-V4-Pro, most intervals between consecutive prefill opportunities accumulate less than 20 MB of decode KV. This occupancy is added to the live weight-prefetch footprint when checking the shared SRAM capacity. HBFlex provisions 40 MB of SRAM per stack, or 240 MB across six stacks, and forces KV writeback when the shared occupancy approaches the provisioned capacity. At the 3 nm technology node [32], a 40 MB SRAM macro with 20% peripheral overhead occupies approximately 8.06 mm², or about 6.7% of a 121 mm² HBF base die [9]. All evaluated designs use the same per-stack SRAM capacity so that comparisons reflect buffer management rather than SRAM provisioning.

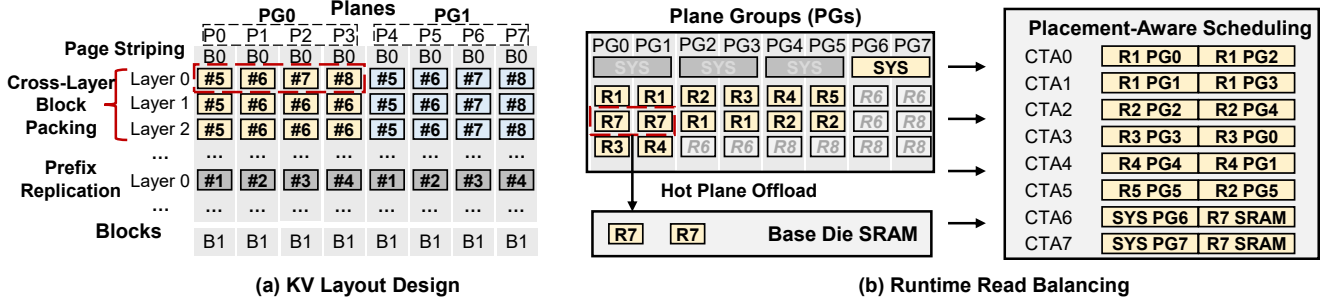


Figure 12. KV layout design and runtime read balancing in HBFlex. (a) Page striping, cross-layer block packing, and prefix replication organize KV cache across planes and blocks. (b) Runtime balancing mitigates hot-plane accesses through selective offloading and placement-aware scheduling.

4.2 Architecture Overview

Figure 11(a) shows the serving runtime and the three physical domains of HBFlex: the GPU compute die, the HBF base die, and the NAND core dies. Prefill and decode execute on the GPU, which issues attention requests through D2D; the base die translates them through its NAND Controller and TSV PHY to selected core-die planes. The base die also contains the SRAM Buffer, while each core die provides NAND planes and local peripheral circuits for selecting array locations and buffering page data. The GPU consequently obtains aggregate plane-level HBF bandwidth without managing NAND commands, plane conflicts, or block reclamation.

Three runtime modules expose the logical control plane. The Paged Logical KV Manager handles logical-page allocation, prefix reuse, request ownership, and validity. The Placement-Aware Read Scheduler orders attention work tiles by expected plane load, and the Write/Erase-Aware Controller tracks dirty state and lifetime information for write and garbage-collection decisions. HBFlex separates policy units through four explicit granularities, as illustrated by the page-group placement in Figure 11(b). A *logical KV page* is the runtime’s allocation and prefix-reuse unit. A *physical stripe* groups G adjacent physical pages across complementary planes without changing the logical page size. A *work tile* is the request-dependent KV subset consumed by one attention CTA. An *erase block* is the NAND reclamation unit whose valid pages must be copied before erase. Mappings among these units allow reuse, placement, scheduling, and reclamation to be optimized independently.

5 HBFlex System Design

5.1 Design Overview

To accommodate dynamic KV cache in HBF while preserving fine-grained logical management, we propose three coordinated mechanisms that address read imbalance, write-read interference, and erase amplification. First, *HBF-aware KV placement and scheduling* coordinates physical page placement with runtime read balancing and attention scheduling,

improving plane utilization across both the active KV working set and individual attention accesses. Second, *window-aware KV writeback* aggregates incremental updates in base-die SRAM and schedules wider writes during sufficiently long compute windows. By consolidating weight reads through prefetching and exploiting prefill computation, it reduces interference between flash programs and foreground reads. Third, *lifetime-aware KV reclamation* combines lifetime-guided block packing with deferred reclamation. It groups pages with similar expected lifetimes within erase blocks and allows more pages to become invalid before GC, reducing valid-page migration and erase amplification.

5.2 HBF-Aware KV Placement and Scheduling

HBFlex combines physical KV layout with runtime read balancing to address both placement and access imbalance. As illustrated in Figure 12, the layout distributes KV pages across planes and provides alternative locations for shared prefixes. The runtime uses this layout to rebalance active KV accesses and schedule attention chunks according to their physical placement.

Plane-Balanced Placement. HBFlex preserves small logical KV pages for fine-grained prefix reuse, while organizing their backing physical pages to expose plane parallelism. We partition HBF planes into *Plane Groups (PGs)* and stripe adjacent physical pages across distinct planes within each group. This layout spreads accesses to neighboring pages across planes without increasing the logical allocation or reuse granularity. HBFlex also replicates frequently shared prefixes, such as system prompts, across multiple PGs. These copies let the runtime serve the same logical pages from different locations, providing flexibility to redirect reads as the active working set changes.

Runtime Read Balancing. Request completion and reactivation can disrupt the balance of the active KV working set. To address this placement imbalance, HBFlex first redirects shared-prefix reads to replicas in less-loaded PGs, without additional data movement. It then caches excess KV pages

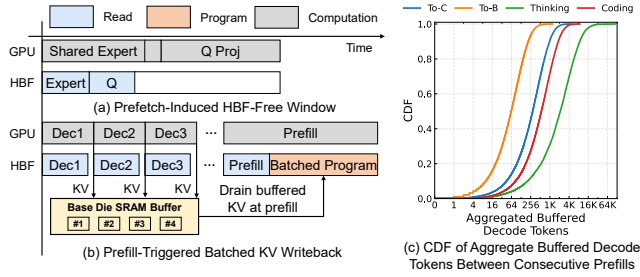


Figure 13. Window-aware KV writeback. (a) Prefetching creates an HBF-free compute window. (b) Batched decode-KV writeback during prefill. (c) CDF of decode tokens buffered between prefills across four Bailian traces.

from the remaining hot planes in base-die SRAM. By offloading only the excess pages, HBFlex reduces load on these planes while limiting SRAM usage and data movement.

Balancing the active working set does not ensure balanced accesses within each attention invocation. HBFlex therefore makes chunk construction and CTA scheduling placement-aware, addressing access imbalance at runtime. For each scheduling wave, it greedily selects KV chunks whose pages collectively cover as many distinct planes as possible. Among candidates with similar plane balance, it favors a chunk from the request previously processed by the CTA, enabling query-vector reuse. This policy improves plane utilization while preserving query locality and the original chunked-attention computation.

5.3 Window-Aware KV Writeback

As discussed in Challenge 2, aggregating incremental KV updates improves write parallelism, but does not by itself prevent interference with foreground reads. HBFlex therefore decouples KV generation from flash programming, buffering updates in base-die SRAM until a suitable writeback opportunity arises. To make these opportunities available, it combines proactive weight prefetching with the longer computation offered by prefill.

Weight reads interleaved with linear operations fragment computation into short intervals. To create longer writeback windows, HBFlex prefetches the weights of consecutive operations into base-die SRAM, as illustrated in Figure 13(a). These operations can then run back-to-back without intervening weight reads, allowing programming to overlap computation. It jointly prefetches the three projections of an expert or the two Q-projection matrices in models with low-rank Q projections. For DeepSeek-V4-Pro, these groups occupy approximately 63 and 106 MB in INT8, respectively. These weights share the 240-MB SRAM budget with buffered KV and other temporary data, subject to the capacity constraint in Section 4.1.

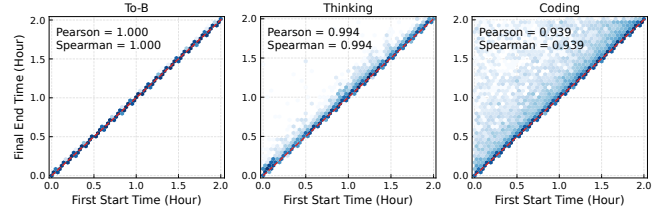


Figure 14. Correlation between request-series start and final end times across three workload traces.

Although prefetching consolidates computation, individual decode stages may still be too short to hide a full program. HBFlex therefore retains newly generated decode KV in SRAM and preferentially drains it during prefill, as illustrated in Figure 13(b). Once the required data have been fetched, the longer prefill computation provides a suitable interval for writeback. This delay also allows updates from multiple decode steps and requests to accumulate into wider programs, improving plane-level write utilization.

Deferring decode writes until prefill requires sufficient SRAM to hold the intervening updates. We assess this demand using the aggregate number of decode tokens generated between consecutive prefill opportunities. As shown in Figure 13(c), most intervals accumulate fewer than 4K tokens across requests. At approximately 5 KB/token for DeepSeek-V4-Pro, this corresponds to less than 20 MB of buffered KV. When prefill opportunities are sparse, shared occupancy may approach capacity before a suitable window appears. HBFlex then forces writeback and temporarily stalls decoding, ensuring bounded buffer usage.

5.4 Lifetime-Aware KV Reclamation

To reduce the erase amplification identified in Challenge 3, HBFlex coordinates lifetime-guided block packing with deferred reclamation. Packing groups KV pages likely to become invalid together, while deferred reclamation allows more invalid pages to accumulate before GC. Both mechanisms reduce the valid data that must be migrated before block erase. KV pages from different Transformer layers of the same request have closely related lifetimes, as they represent the same token history and are typically evicted together. HBFlex exploits this relationship by packing cross-layer KV pages into the same flash blocks whenever possible, as illustrated in Figure 12(a).

Beyond this within-request correlation, HBFlex uses the root request’s start time to estimate lifetime similarity across request series. As shown in Figure 14, series that start at similar times tend to have similar final end times. HBFlex therefore assigns lifetime-interval labels based on root-request start times and propagates them to descendant requests and their prefill/decode KV pages. GC-relocated pages retain these labels to preserve lifetime grouping.

When allocating KV pages, HBFlex first looks for a partially filled block with a matching lifetime-interval label. If none is available, it opens a free block for that interval or uses a block with the closest label. This policy groups pages with similar expected lifetimes without requiring each request to occupy dedicated blocks, preserving flexibility for plane-level striping.

Lifetime-guided packing improves the choice of pages within a block, but reclaiming the block too early can still require unnecessary migration. HBFlex therefore separates KV eviction from physical block reclamation. It triggers eviction at 90% memory utilization and GC at 95%, allowing additional pages to become invalid before blocks are reclaimed. This interval gives GC more opportunities to select blocks with fewer valid pages, further reducing migration overhead.

6 Experimental Evaluation

6.1 Experimental Setup

Models and Workloads. We evaluate HBFlex on DeepSeek-V4-Pro [6], DeepSeek-V3 [5], Hunyuan3 [28], and Qwen3-235B [34]. DeepSeek-V4-Pro uses two-way tensor parallelism (TP) combined with four-way data parallelism (DP), while the other models use four-way DP. Our workloads include four Alibaba Bailian production traces [30]: To-B, To-C, Coding, and Thinking. We also collect SWE-bench [12] traces by running tasks with Qwen3 and recording the inter-request intervals within each task. Varying task concurrency allows us to evaluate performance under different serving loads.

Hardware Configuration. We model H200-class accelerators [19] with a common per-GPU compute capability and memory-package area budget. The full-HBF configuration contains six stacks per GPU. A full HBF stack contains 16 dies, with 32 independent planes per die. Each plane contains 512 blocks, and each block contains 512 pages of 4 KiB. The read and program latencies are set to 4 μ s and 75 μ s, respectively. We configure each HBF stack with 488 GB/s read bandwidth and 27.2 GB/s write bandwidth. Each HBF stack is equipped with a 40 MB base-die SRAM buffer for latency hiding and temporary data buffering. We model an HBM3e stack with 8 DRAM dies, 24 GB capacity, and 0.8 TB/s bandwidth [11].

Baselines. We compare HBFlex with representative HBF-based LLM serving designs, including H3, FlashAccel, and an integrated-stack design (IS). H3 adopts a cascaded HBM/HBF organization, where read-mostly data are placed in HBF while frequently updated states remain in HBM. FlashAccel is evaluated under both co-located and cascaded organizations. Following its original design, the co-located configuration uses five HBF stacks and one HBM stack. We also evaluate an integrated HBM/HBF stack organization, where HBM and HBF dies are combined within the same stack.

Table 1. Comparison of HBF-based LLM serving systems.

System	Architecture	KV Location	Key Feature
H3 [9]	Cascaded	HBM+HBF	Prefix KV in HBF
FA-CLI [31]	Co-located	HBM+HBF	Layout optimization
FA-CSI [31]	Cascaded	HBM+HBF	Layout optimization
IS [38]	Integrated Stack	HBM	Stack integration
HBFlex	Full HBF	HBF	Flexible Management

For cascaded designs, we normalize the memory resources to the same package-area budget. In the normalized configuration, one quarter of the memory budget is assigned to HBM and three quarters to HBF. Relative to full stacks, HBM capacity is scaled to one quarter, while HBF capacity and read/write bandwidth are scaled to three quarters. HBM bandwidth remains unchanged because the modeled external interface width is preserved. HBFlex uses six full HBF stacks per GPU, with base-die SRAM supporting dynamic KV management. All evaluated designs use the same per-stack SRAM capacity.

6.2 End-to-End Performance

We first evaluate serving latency using four Alibaba Bailian traces across four models, as shown in Figure 15. We report geometric-mean speedups with equal weight for each model-trace combination. HBFlex achieves time to first token (TTFT) speedups of 1.20 $\times$ and 1.33 $\times$ over FA-CLI and FA-CSI, respectively, with corresponding time per output token (TPOT) speedups of 1.21 $\times$ and 1.35 $\times$. Compared with H3, the average speedups reach 22.65 $\times$ in TTFT and 1.89 $\times$ in TPOT. The larger TTFT gain reflects reduced admission queueing under KV-capacity pressure, in addition to faster request processing.

To examine how these benefits vary with serving load, we use SWE-bench traces with different numbers of concurrent tasks, as illustrated in Figure 16. At lower concurrency, the smaller active KV working set makes weight loading a larger component of memory service time. Under the same package-area budget, HBFlex dedicates more resources to HBF and exposes more planes for parallel weight reads. At the lowest plotted concurrency, HBFlex achieves average throughput speedups of 1.31 $\times$ over FA-CLI and 1.47 $\times$ over FA-CSI. These gains primarily reflect faster weight loading, before KV capacity and access efficiency become influential.

As concurrency increases, KV accesses account for a larger share of memory traffic, making KV management increasingly important. Compared with FlashAccel, which also stores KV in HBF, HBFlex gains additional benefits from placement-aware attention scheduling and coordinated write-back, improving read utilization while reducing write-read interference. At the highest plotted concurrency, the average throughput speedups over FA-CLI and FA-CSI increase to 1.48 $\times$ and 1.58 $\times$, respectively. Compared with H3, its larger HBF-resident KV capacity allows more requests to remain

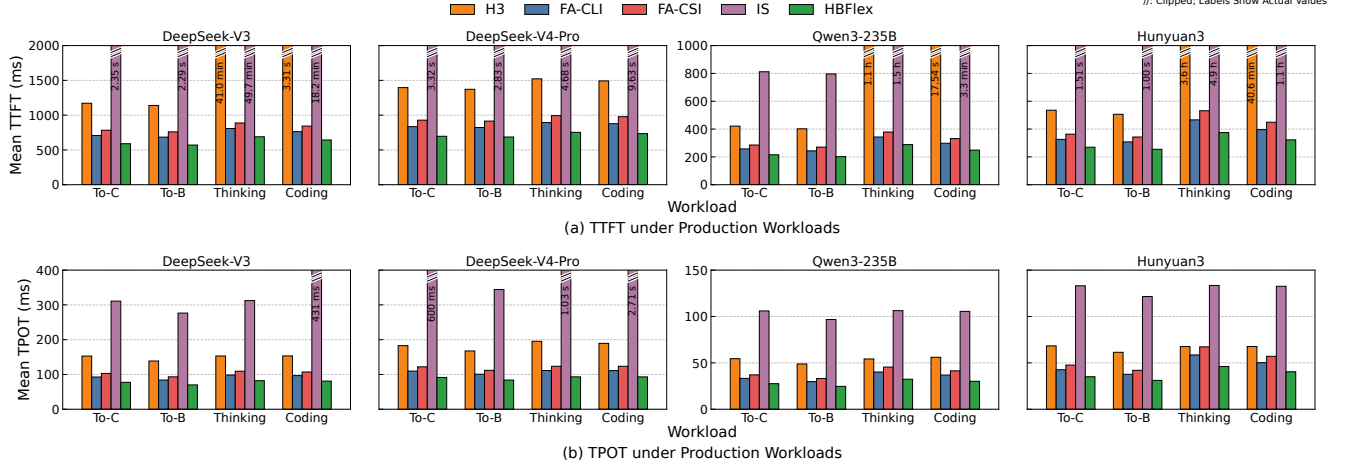


Figure 15. Serving latency across four models and four Alibaba Bailian traces: (a) mean TTFT and (b) mean TPOT. Labels above clipped bars indicate actual values.

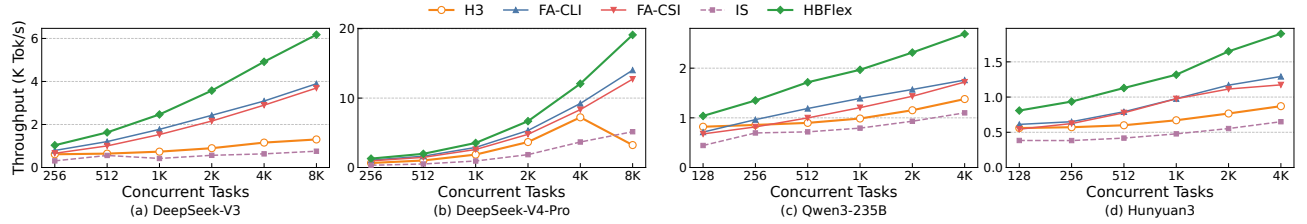


Figure 16. Serving throughput across four models as the number of concurrent tasks increases on SWE-bench traces.

Table 2. Throughput speedups from serving simulation with cumulatively enabled HBFlex techniques, normalized to Naive HBF for each model.

Cumulative configuration	DeepSeek-V3	Qwen3-235B
Naive HBF	1.00×	1.00×
+ Plane-Balanced Placement	2.94×	2.59×
+ Runtime Read Balancing	4.08×	4.25×
+ Window-Aware KV Writeback	4.10×	4.27×
+ Lifetime-Aware KV Reclamation	4.15×	4.32×

active concurrently, relieving the admission constraints imposed by HBM-resident KV. Across the four models, the average throughput speedup over H3 increases from 1.56× at the lowest plotted concurrency to 3.30× at the highest. These results illustrate how the benefits extend from faster weight loading at low concurrency to more efficient KV access and greater serving concurrency at higher loads.

6.3 Ablation Study

We evaluate the cumulative throughput benefits of HBFlex through serving simulation with 4,096 concurrent SWE-bench tasks across four data-parallel GPUs. Starting from

Naive HBF with immediate block-first writes, we progressively enable each technique while retaining the preceding optimizations.

As reported in Table 2, most of the throughput improvement comes from optimizing KV reads. Plane-balanced placement and runtime read balancing together achieve speedups of 4.08× for DeepSeek-V3 and 4.25× for Qwen3-235B. Placement also improves program parallelism, so this cumulative gain includes benefits to both reads and writes.

Window-aware KV writeback provides a smaller additional gain, increasing the normalized speedup by approximately 0.02×. The relatively small per-token KV footprints of the evaluated models limit write traffic, leaving less write overhead to eliminate. Lifetime-aware KV reclamation adds approximately 0.05×, bringing the overall speedups to 4.15× and 4.32×. Its primary benefit is improved flash endurance through fewer valid-page migrations and block erases, while its contribution to throughput is more modest.

6.4 Performance Analysis

KV Placement and Scheduling. We evaluate KV read efficiency using read-wave amplification, defined as the ratio of actual read cycles to the theoretical minimum under perfect plane utilization. As illustrated in Figure 17, we compare three configurations across four concurrency levels for

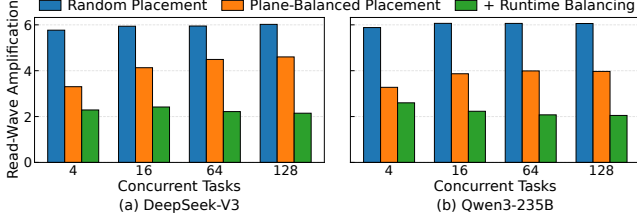


Figure 17. Effect of HBF-aware KV placement and runtime balancing on read-wave amplification.

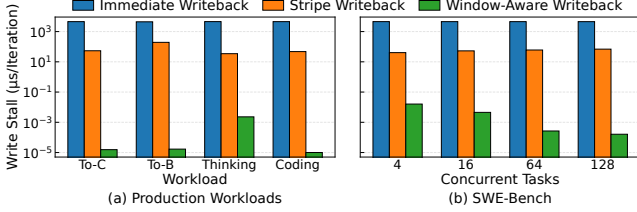


Figure 18. Effect of window-aware writeback on KV write stall.

DeepSeek-V3 and Qwen3-235B. Averaged across all configurations of model and concurrency, plane-balanced placement reduces amplification from $5.97\times$ to $3.95\times$, while runtime balancing further lowers it to $2.25\times$. Together, these mechanisms achieve an average reduction of 62.21% relative to random placement.

Balanced placement distributes stored KV pages across planes, but each attention wave may still access only a subset of planes, leaving residual amplification. Runtime balancing reduces these access conflicts through selective offloading and placement-aware scheduling. The additional improvement highlights the need to balance both stored pages and runtime accesses.

Window-Aware KV Writeback. We measure write-induced critical-path stall per serving iteration for DeepSeek-V4-Pro across four Bailian traces and four SWE-bench concurrency levels. As shown in Figure 18, the average stall decreases from 4.54 ms with immediate writeback to 69.12 μs with stripe writeback, and further to approximately 0.003 μs with window-aware writeback.

Stripe writeback amortizes program overhead by combining small KV updates into larger writes, but programs can still block subsequent reads. Window-aware writeback adds weight prefetching and exploits prefill compute windows to overlap programming with computation, nearly eliminating stalls without shortening physical program latency. The end-to-end gain remains modest when KV writes account for only a small fraction of serving time.

Lifetime-Aware KV Reclamation. We evaluate lifetime-aware KV reclamation using write amplification (WAF) [10] across four Bailian traces for DeepSeek-V3 and Qwen3-235B.

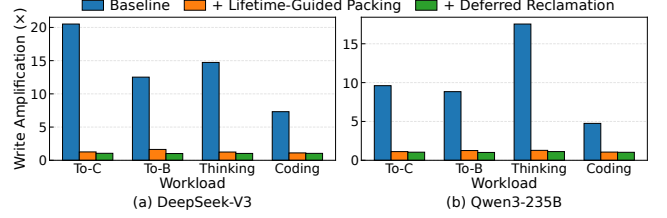


Figure 19. Effect of lifetime-guided packing and deferred reclamation on garbage-collection write amplification.

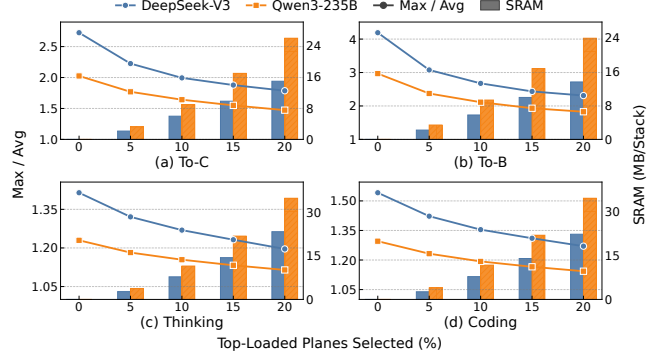


Figure 20. Trade-off between hot-plane offloading and SRAM usage across four Bailian traces. Lines show the maximum-to-average plane load; bars show SRAM usage per stack.

As illustrated in Figure 19, average WAF decreases from $11.98\times$ in the baseline to $1.24\times$ with lifetime-guided packing, and further to $1.04\times$ with deferred reclamation. The complete scheme achieves an average per-scenario reduction of 89.39% relative to the baseline.

Mixing KV lifetimes leaves valid pages in GC victims, requiring migration before erase. Plane-group organization and lifetime-guided packing concentrate invalidations within reclamation units, while deferred reclamation lets more pages become invalid before GC. Together, they reduce copying and bring WAF close to one. This limits extra writes and associated erases, primarily benefiting endurance, alongside the modest throughput gain observed in the cumulative ablation.

6.5 Hot-Plane Offload and SRAM Usage

We examine the trade-off between read balance and SRAM usage by varying the fraction of top-loaded planes selected for offloading. The selected fraction ranges from 0% to 20% under four Alibaba Bailian traces. For each selected plane, excess KV pages are cached in base-die SRAM. As illustrated in Figure 20, we measure the maximum-to-average plane load and the SRAM footprint per stack for DeepSeek-V3 and Qwen3-235B.

Selecting more planes reduces load imbalance but increases SRAM usage. The improvement is more pronounced on To-C and To-B, where the initial imbalance is higher. Beyond 10%,

the curves flatten while SRAM usage continues to grow. At 10%, the offload footprint is approximately 6–12 MB per stack across the evaluated configurations. Increasing the fraction to 20% raises this footprint to approximately 14–35 MB per stack. This leaves less SRAM for weight prefetching, KV writeback, and GC migration. We therefore select the top-loaded 10% of planes as a trade-off between read balance and SRAM usage.

7 Conclusion

This paper presented HBFlex, a cross-layer memory system for serving LLMs with dynamic KV cache in a full-HBF organization. Realizing HBF’s potential requires addressing placement and access imbalance, write–read interference, and GC amplification from mixed KV lifetimes. HBFlex combines HBF-aware placement and attention scheduling, window-aware writeback, and lifetime-aware reclamation to coordinate KV reads, writes, and erases. Supported by base-die SRAM and co-located prefill and decode, these mechanisms preserve fine-grained prefix reuse while adapting KV management to HBF’s physical constraints. Across the evaluated configurations, HBFlex achieves average throughput speedups of up to 1.58 $\times$ over FlashAccel and 3.30 $\times$ over H3, benefiting from higher HBF bandwidth and more efficient KV-cache management. These results demonstrate the benefits of coordinating KV management to exploit full-HBF memory resources.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 111–126.
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2024. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 315–329.
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Amar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [4] Tri Dao. 2024. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, Vol. 2024. 35549–35562.
- [5] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] <https://arxiv.org/abs/2412.19437>
- [6] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 [cs.CL] <https://arxiv.org/abs/2606.19348>
- [7] Kaustubh Deshpande, Ved Sirdeshmukh, Johannes Baptist Mols, Lifeng Jin, Ed-Yeremai Hernandez-Cardona, Dean Lee, Jeremy Kritiz, Willow E. Primack, Summer Yue, and Chen Xing. 2025. MultiChallenge: A Realistic Multi-Turn Conversation Evaluation Benchmark Challenging to Frontier LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*. 18632–18702.
- [8] Arijit Dirik and Bruce Jacob. 2009. The Performance of PC Solid-State Disks (SSDs). In *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*. 278–289. doi:10.1145/1555754.1555790
- [9] Minh Ha, Euiseok Kim, and Hoshik Kim. 2026. H3: Hybrid Architecture Using High Bandwidth Memory and High Bandwidth Flash for Cost-Efficient LLM Inference. *IEEE Computer Architecture Letters* 25, 1 (2026), 49–52. doi:10.1109/LCA.2026.3660969
- [10] Xiao-Yu Hu, Evangelos Eleftheriou, Robert Haas, Ilias Iliadis, and Beat Poff. 2009. Write amplification analysis in flash-based solid state drives. In *Proceedings of the 2nd ACM International Systems and Storage Conference (SYSTOR ’09)*. ACM, New York, NY, USA, 1–9. doi:10.1145/1534530.1534544
- [11] JEDEC Solid State Technology Association. 2023. *High Bandwidth Memory (HBM3/HBM3e) DRAM Standard*. JEDEC Standard JESD238A. JEDEC. <https://www.jedec.org/standards-documents/docs/jesd238a>
- [12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165
- [14] Kwanhee Kyung, Yeon Ji Moon, Juhwan Cho, and Jung Ho Ahn. 2026. High-Bandwidth Flash for KV Caches: Endurance and Performance Implications. *IEEE Computer Architecture Letters* 25, 1 (2026), 210–213. doi:10.1109/LCA.2026.3695938
- [15] Hyuni Lee, Haeyeon Kim, JungHyun Lee, Inyoung Choi, Jaehun Lee, Hyowon An, Chaemin Yang, and Joungho Kim. 2025. Mem-Village: A Hybrid HBF-HBM Architecture on Glass for Ultra-Scale AI Inference Systems. In *2025 IEEE Electrical Design of Advanced Packaging and Systems*. IEEE, Piscataway, NJ, USA, 1–3. doi:10.1109/EDAPS66187.2025.11411753
- [16] Zhuoran Li, Zhuohang Bian, Xin Huang, Yibo Zhao, Guangyu Sun, and Youwei Zhuo. 2026. HBF Sucks! A Full-Stack Characterization of High-Bandwidth Flash for KV-Centric LLM Serving. arXiv:2608.11668 [cs.AR] doi:10.48550/arXiv.2608.11668
- [17] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Boston, MA, USA, 663–679. <https://arxiv.org/abs/2302.11665>
- [18] Xiaoyu Ma and David Patterson. 2026. Challenges and Research Directions for Large Language Model Inference Hardware. *Computer* 59, 5 (2026), 55–64. doi:10.1109/MC.2026.3652916
- [19] NVIDIA. 2023. NVIDIA H200 Tensor Core GPU. Product specification, accessed 2026-09-08. <https://www.nvidia.com/en-us/data-center/h200/>
- [20] Open Compute Project. 2023. *OCF HBF Architecture Specification v0.7.0 FINAL*. Technical Specification v0.7.0. Open Compute Project. <https://www.opencompute.org/>
- [21] Junho Park, Hyowon An, Haeseok Suh, Youngsu Yoon, Hyuni Lee, and Joungho Kim. 2026. HBM-HBF-Centric Memory Pooling Architecture with Custom Base Die for Terabyte-Scale LLM Inference. *IEEE Computer Architecture Letters* 25, 2 (2026), 259–262. doi:10.1109/LCA.2026.3703982

- [22] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. 1–22.
- [23] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*.
- [24] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2024. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles*. 294–308.
- [25] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies*.
- [26] Dowon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. 2026. Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges. *IEEE Computer Architecture Letters* 25, 2 (2026), 1–4. doi:10.1109/LCA.2026.3705817
- [27] Yutao Sun et al. 2024. Hunyuan-Large: An Open-Source MoE Model with 52 Billion Activated Parameters by Tencent. arXiv:2411.02265 [cs.CL] <https://arxiv.org/abs/2411.02265>
- [28] Tencent Hy Team. 2026. Hy3. <https://huggingface.co/tencent/Hy3>. Hugging Face model repository, accessed September 10, 2026.
- [29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023).
- [30] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2026. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. arXiv:2506.02634 [cs.DC] <https://arxiv.org/abs/2506.02634>
- [31] Xinyu Wang, Yalong Xue, Xiaotian Sun, Xiaoyu Zhang, Chunmeng Dou, Xueqi Li, and Xiaoming Chen. 2026. FlashAccel: Leveraging High-Bandwidth Flash for High-Throughput LLM Inference. arXiv:2607.10186 [cs.AR] doi:10.48550/arXiv.2607.10186
- [32] Shien-Yang Wu et al. 2022. A 3nm CMOS FinFlex Technology with Enhanced Power Efficiency and Performance for Mobile SoC and High Performance Computing Applications. In *2022 IEEE International Electron Devices Meeting*. IEEE, Piscataway, NJ, USA, 27.1.1–27.1.4. <https://ieeexplore.ieee.org/document/10019483>
- [33] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*.
- [34] An Yang et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*.
- [36] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).
- [37] Zihao Yi, Jiarui Ouyang, Zhe Xu, Yuwen Liu, Tianhao Liao, Haohao Luo, and Ying Shen. 2025. A Survey on Recent Advances in LLM-Based Multi-Turn Dialogue Systems. *Comput. Surveys* 58, 6 (2025), 1–38.
- [38] Yihan Yin, Yilun Zhao, Zhixin Yun, Guanying Wu, Feng Zhu, Kai Tao, Shu Li, Fei Huang, Zhe Zhang, Shuangchen Li, and Hongzhong Zheng. 2026. Potential Applications of HBF in LLM Serving Systems. arXiv:2608.13127 [cs.AR] <https://arxiv.org/abs/2608.13127>
- [39] Zichang Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 34661–34673.
- [40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Haoteng Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, Vol. 36. 46595–46623.
- [41] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 62557–62583. <https://arxiv.org/abs/2312.07104>
- [42] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Santa Clara, CA, USA, 193–210. <https://arxiv.org/abs/2401.09670>